NAME:KISHORE KUMAR V

REG NO:192521371

COURSE CODE:ITA0613

EXP4: Artificial Neural Network using Backpropagation

Session Topic: Build an Artificial Neural Network by implementing the Backpropagation algorithm and test the same using appropriate data sets.

Question 1: Backpropagation Neural Network for the XOR Problem

Aim:

To build an Artificial Neural Network with one hidden layer, train it using the backpropagation algorithm in Python, and test it on the XOR data set.

Algorithm:

1. Start. Load the XOR input patterns and their target outputs.
2. Initialise the weights with small random values and the biases with zero.
3. Forward pass: compute the hidden layer output and the final output using the sigmoid activation function.
4. Calculate the error between the target output and the actual output.
5. Backward pass: compute the error terms (deltas) for the output layer and then for the hidden layer.
6. Update all weights and biases: w = w - learning rate × gradient.

7. Repeat steps 3 to 6 for the given number of epochs, then test the network on all input patterns.
8. Stop.

Data Set:

XOR training data set (4 records):

Input X1	Input X2	Target Output
0	0	0
0	1	1
1	0	1
1	1	0

Python Code:

```
import numpy as np

# XOR training data
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
y = np.array([[0], [1], [1], [0]])

np.random.seed(1)
```

```
W1 = np.random.randn(2, 4); b1 = np.zeros((1, 4))   # input
-> hidden
W2 = np.random.randn(4, 1); b2 = np.zeros((1, 1))   #
hidden -> output
lr = 0.5

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

for epoch in range(1, 10001):
    # forward pass
    h = sigmoid(X @ W1 + b1)
    out = sigmoid(h @ W2 + b2)
    # backward pass (backpropagation)
    d_out = (out - y) * out * (1 - out)
    d_h = (d_out @ W2.T) * h * (1 - h)
    W2 -= lr * h.T @ d_out;  b2 -= lr * d_out.sum(axis=0,
keepdims=True)
    W1 -= lr * X.T @ d_h;    b1 -= lr * d_h.sum(axis=0,
keepdims=True)
    if epoch % 2000 == 0:
```

```
        print(f"Epoch {epoch:5d}  Error = {np.mean((y - out)
** 2):.6f}")

print("\nTesting the trained network:")
for x, p in zip(X, out):
    print(f"Input {x}  Output = {p[0]:.4f}  Predicted =
{int(p[0] > 0.5)}")
```

Output:

```
Epoch  2000  Error = 0.002303
Epoch  4000  Error = 0.000808
Epoch  6000  Error = 0.000472
Epoch  8000  Error = 0.000328
Epoch 10000  Error = 0.000250

Testing the trained network:
Input [0 0]  Output = 0.0085  Predicted = 0
Input [0 1]  Output = 0.9855  Predicted = 1
Input [1 0]  Output = 0.9837  Predicted = 1
Input [1 1]  Output = 0.0213  Predicted = 0
```

Result:

The neural network was trained successfully using backpropagation. The error fell to 0.00025 after 10000 epochs and the network classified all 4 XOR patterns correctly.

Question 2: Backpropagation Neural Network for Student Pass / Fail Prediction

Aim:

To train an Artificial Neural Network using the backpropagation algorithm to predict whether a student will pass or fail from the study hours and attendance, and to test it with a new sample.

Algorithm:

1. Start. Load the student data set and normalise the input values.
2. Initialise the weights randomly and the biases with zero (2 input, 5 hidden and 1 output neuron).
3. Forward pass: calculate the hidden and output layer activations using the sigmoid function.
4. Compute the output error and the delta values of the output and hidden layers.
5. Update the weights and biases using the gradient descent rule.

6. Repeat steps 3 to 5 for 5000 epochs and print the error at regular intervals.
7. Calculate the training accuracy and predict the result for a new student record.
8. Stop.

Data Set:

Student data set (12 records):

Study Hours	Attendance (%)	Result (1 = Pass, 0 = Fail)
1	50	0
2	55	0
2	70	0
3	60	0
3	80	1
4	65	0
5	90	1
6	75	1
7	95	1

Study Hours	Attendance (%)	Result (1 = Pass, 0 = Fail)
8	85	1
1	90	0
8	40	0

Python Code:

```
import numpy as np

# Student data: [study hours, attendance %] -> pass (1) / fail (0)
X = np.array([[1, 50], [2, 55], [2, 70], [3, 60], [3, 80], [4, 65],
              [5, 90], [6, 75], [7, 95], [8, 85], [1, 90], [8, 40]], float)
y = np.array([[0], [0], [0], [0], [1], [0], [1], [1], [1], [1], [0], [0]], float)

mean, std = X.mean(axis=0), X.std(axis=0)
Xn = (X - mean) / std                    # normalise the inputs
```

```
np.random.seed(3)
W1 = np.random.randn(2, 5) * 0.5; b1 = np.zeros((1, 5))
W2 = np.random.randn(5, 1) * 0.5; b2 = np.zeros((1, 1))
lr = 0.3

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

for epoch in range(1, 5001):
    h = sigmoid(Xn @ W1 + b1)
    out = sigmoid(h @ W2 + b2)
    d_out = (out - y) * out * (1 - out)
    d_h = (d_out @ W2.T) * h * (1 - h)
    W2 -= lr * h.T @ d_out;  b2 -= lr * d_out.sum(axis=0,
keepdims=True)
    W1 -= lr * Xn.T @ d_h;   b1 -= lr * d_h.sum(axis=0,
keepdims=True)
    if epoch % 1000 == 0:
        print(f"Epoch {epoch:4d}  Error = {np.mean((y - out)
** 2):.5f}")
```

```
pred = (out > 0.5).astype(int)
print("\nTraining accuracy = %.2f%%" % (100 *
np.mean(pred == y)))

new = (np.array([[6, 80]], float) - mean) / std
p = sigmoid(sigmoid(new @ W1 + b1) @ W2 + b2)[0, 0]
print("New student [6 hours, 80%% attendance]:
probability = %.4f" % p)
print("Prediction:", "Pass" if p > 0.5 else "Fail")
```

Output:

```
Epoch 1000  Error = 0.00456
Epoch 2000  Error = 0.00132
Epoch 3000  Error = 0.00069
Epoch 4000  Error = 0.00044
Epoch 5000  Error = 0.00032

Training accuracy = 100.00%
New student [6 hours, 80% attendance]: probability =
1.0000
```

Prediction: Pass

Result:

The network was trained with backpropagation and reached 100% training accuracy with an error of 0.00032. The new student (6 hours, 80% attendance) was predicted to Pass.

Question 3: Backpropagation Neural Network for Iris Flower Classification

Aim:

To implement the backpropagation algorithm for a multi-class neural network and test it on the Iris data set to classify flowers into three species.

Algorithm:

1. Start. Load the Iris data set and split it into training (70%) and testing (30%) sets.
2. Standardise the input features and convert the class labels to one-hot form.
3. Initialise the weights randomly (4 input, 8 hidden and 3 output neurons).
4. Forward pass: compute the hidden and output layer outputs using the sigmoid function.
5. Backward pass: calculate the output error and propagate it back to the hidden layer.
6. Update the weights and biases; repeat for 2000 epochs.

7. Find the training and testing accuracy and classify a new flower sample.
8. Stop.

Data Set:

Iris data set (150 records, 10 sample records shown):

Sepal Length	Sepal Width	Petal Length	Petal Width	Class
5.1	3.5	1.4	0.2	setosa
4.9	3	1.4	0.2	setosa
4.7	3.2	1.3	0.2	setosa
7	3.2	4.7	1.4	versicolor
6.4	3.2	4.5	1.5	versicolor
6.9	3.1	4.9	1.5	versicolor
6.3	3.3	6	2.5	virginica
5.8	2.7	5.1	1.9	virginica
7.1	3	5.9	2.1	virginica
5.9	3	5.1	1.8	virginica

Python Code:

```
import numpy as np
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

iris = load_iris()
X, y = iris.data, iris.target
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3,
                                      random_state=42, stratify=y)
mean, std = Xtr.mean(axis=0), Xtr.std(axis=0)
Xtr, Xte = (Xtr - mean) / std, (Xte - mean) / std
T = np.eye(3)[ytr]                          # one-hot target

np.random.seed(0)
W1 = np.random.randn(4, 8) * 0.5; b1 = np.zeros((1, 8))
W2 = np.random.randn(8, 3) * 0.5; b2 = np.zeros((1, 3))
lr = 0.5

def sigmoid(z):
    return 1 / (1 + np.exp(-z))
```

```
for epoch in range(1, 2001):
    h = sigmoid(Xtr @ W1 + b1)
    out = sigmoid(h @ W2 + b2)
    d_out = (out - T) * out * (1 - out)
    d_h = (d_out @ W2.T) * h * (1 - h)
    W2 -= lr * h.T @ d_out / len(Xtr);  b2 -= lr *
d_out.mean(axis=0, keepdims=True)
    W1 -= lr * Xtr.T @ d_h / len(Xtr);  b1 -= lr *
d_h.mean(axis=0, keepdims=True)
    if epoch % 500 == 0:
        print(f"Epoch {epoch:4d}  Error = {np.mean((T - out)
** 2):.5f}")

def predict(A):
    return np.argmax(sigmoid(sigmoid(A @ W1 + b1) @
W2 + b2), axis=1)

print("\nTraining accuracy = %.2f%%" % (100 *
np.mean(predict(Xtr) == ytr)))
print("Testing accuracy  = %.2f%%" % (100 *
np.mean(predict(Xte) == yte)))
```

```
new = (np.array([[6.0, 2.9, 4.5, 1.5]]) - mean) / std
print("New sample [6.0, 2.9, 4.5, 1.5] ->",
iris.target_names[predict(new)[0]])
```

Output:

```
Epoch  500  Error = 0.05256
Epoch 1000  Error = 0.02365
Epoch 1500  Error = 0.01528
Epoch 2000  Error = 0.01191

Training accuracy = 98.10%
Testing accuracy  = 91.11%
New sample [6.0, 2.9, 4.5, 1.5] -> versicolor
```

Result:

The network was trained using backpropagation and gave a training accuracy of 98.10% and a testing accuracy of 91.11%. The new sample [6.0, 2.9, 4.5, 1.5] was classified as versicolor.

Question 4: Backpropagation Neural Network for Half Adder Implementation

Aim:

To train a neural network with two output neurons using the backpropagation algorithm so that it behaves like a half adder (Sum and Carry).

Algorithm:

1. Start. Load the half adder truth table (inputs A, B and outputs Sum, Carry).
2. Initialise the weights randomly (2 input, 4 hidden and 2 output neurons) and the biases with zero.
3. Forward pass: compute the hidden layer and the two output values using the sigmoid function.
4. Calculate the error for each output neuron.
5. Backward pass: find the delta of the output and hidden layers.
6. Update the weights and biases; repeat for 10000 epochs.

7. Test the trained network for all four input combinations and compare with the truth table.
8. Stop.

Data Set:

Half adder data set (4 records):

Input A	Input B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Python Code:

```
import numpy as np

# Half adder: inputs A, B -> outputs Sum, Carry
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
y = np.array([[0, 0], [1, 0], [1, 0], [0, 1]])

np.random.seed(2)
```

```
W1 = np.random.randn(2, 4); b1 = np.zeros((1, 4))
W2 = np.random.randn(4, 2); b2 = np.zeros((1, 2))
lr = 0.5

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

for epoch in range(1, 10001):
    h = sigmoid(X @ W1 + b1)
    out = sigmoid(h @ W2 + b2)
    d_out = (out - y) * out * (1 - out)
    d_h = (d_out @ W2.T) * h * (1 - h)
    W2 -= lr * h.T @ d_out;  b2 -= lr * d_out.sum(axis=0,
keepdims=True)
    W1 -= lr * X.T @ d_h;    b1 -= lr * d_h.sum(axis=0,
keepdims=True)
    if epoch % 2500 == 0:
        print(f"Epoch {epoch:5d}  Error = {np.mean((y - out)
** 2):.6f}")

print("\nA B | Sum  Carry | Predicted (Sum, Carry)")
```

```
for x, t, p in zip(X, y, out):
    print(f"{x[0]} {x[1]} |  {t[0]}     {t[1]}   |
({round(p[0])}, {round(p[1])})"
          f"   raw = [{p[0]:.3f}, {p[1]:.3f}]")
```

Output:

```
Epoch  2500  Error = 0.000720
Epoch  5000  Error = 0.000301
Epoch  7500  Error = 0.000186
Epoch 10000  Error = 0.000134


A B | Sum  Carry | Predicted (Sum, Carry)
0 0 |  0     0   |  (0, 0)   raw = [0.011, 0.005]
0 1 |  1     0   |  (1, 0)   raw = [0.987, 0.009]
1 0 |  1     0   |  (1, 0)   raw = [0.988, 0.010]
1 1 |  0     1   |  (0, 1)   raw = [0.015, 0.986]
```

Result:

The neural network learned the half adder truth table using backpropagation. The error was 0.000134 after 10000 epochs and both Sum and Carry were predicted correctly for all four inputs.

Question 5: Backpropagation Neural Network for Breast Cancer Classification

Aim:

To build and test an Artificial Neural Network using the backpropagation algorithm on the Breast Cancer data set to classify tumours as malignant or benign.

Algorithm:

1. Start. Load the Breast Cancer data set and split it into training (75%) and testing (25%) sets.
2. Standardise the 30 input features.
3. Initialise the weights randomly (30 input, 10 hidden and 1 output neuron).
4. Forward pass: compute the hidden and output layer activations using the sigmoid function.
5. Backward pass: calculate the deltas and update the weights and biases by gradient descent.
6. Repeat steps 4 and 5 for 1000 epochs.
7. Predict the test data, then calculate the accuracy and the confusion matrix.

8. Stop.

Data Set:

Breast Cancer data set (569 records, 30 features; first 8 records with 4 features shown):

Mean Radius	Mean Texture	Mean Perimeter	Mean Area	Class
17.99	10.38	122.8	1001	malignant
20.57	17.77	132.9	1326	malignant
19.69	21.25	130	1203	malignant
11.42	20.38	77.58	386.1	malignant
20.29	14.34	135.1	1297	malignant
12.45	15.7	82.57	477.1	malignant
18.25	19.98	119.6	1040	malignant
13.71	20.83	90.2	577.9	malignant

Python Code:

```
import numpy as np
from sklearn.datasets import load_breast_cancer
```

```
from sklearn.model_selection import train_test_split

data = load_breast_cancer()
X, y = data.data, data.target.reshape(-1, 1)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.25,
                                      random_state=1, stratify=y)
mean, std = Xtr.mean(axis=0), Xtr.std(axis=0)
Xtr, Xte = (Xtr - mean) / std, (Xte - mean) / std

np.random.seed(0)
W1 = np.random.randn(30, 10) * 0.3; b1 = np.zeros((1, 10))
W2 = np.random.randn(10, 1) * 0.3;  b2 = np.zeros((1, 1))
lr = 0.5

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

for epoch in range(1, 1001):
    h = sigmoid(Xtr @ W1 + b1)
    out = sigmoid(h @ W2 + b2)
    d_out = (out - ytr) * out * (1 - out)
```

```
    d_h = (d_out @ W2.T) * h * (1 - h)
    W2 -= lr * h.T @ d_out / len(Xtr);  b2 -= lr *
d_out.mean(axis=0, keepdims=True)
    W1 -= lr * Xtr.T @ d_h / len(Xtr);  b1 -= lr *
d_h.mean(axis=0, keepdims=True)
    if epoch % 250 == 0:
        print(f"Epoch {epoch:4d}  Error = {np.mean((ytr -
out) ** 2):.5f}")

pred = (sigmoid(sigmoid(Xte @ W1 + b1) @ W2 + b2) >
0.5).astype(int)
print("\nTesting accuracy = %.2f%%" % (100 *
np.mean(pred == yte)))
tp = np.sum((pred == 1) & (yte == 1)); tn = np.sum((pred
== 0) & (yte == 0))
fp = np.sum((pred == 1) & (yte == 0)); fn = np.sum((pred
== 0) & (yte == 1))
print("Confusion matrix (rows = actual, cols = predicted):")
print([[int(tn), int(fp)], [int(fn), int(tp)]])
print("Sample 0 of test set ->", data.target_names[pred[0,
0]],
```

```
"(actual:", data.target_names[yte[0, 0]] + ")")
```

Output:

```
Epoch  250  Error = 0.03254
Epoch  500  Error = 0.02139
Epoch  750  Error = 0.01734
Epoch 1000  Error = 0.01512


Testing accuracy = 97.20%
Confusion matrix (rows = actual, cols = predicted):
[[51, 2], [2, 88]]
Sample 0 of test set -> malignant (actual: malignant)
```

Result:

The network was trained with backpropagation and achieved a testing accuracy of 97.20% (2 false positives and 2 false negatives). The sample tumour was correctly classified as malignant.